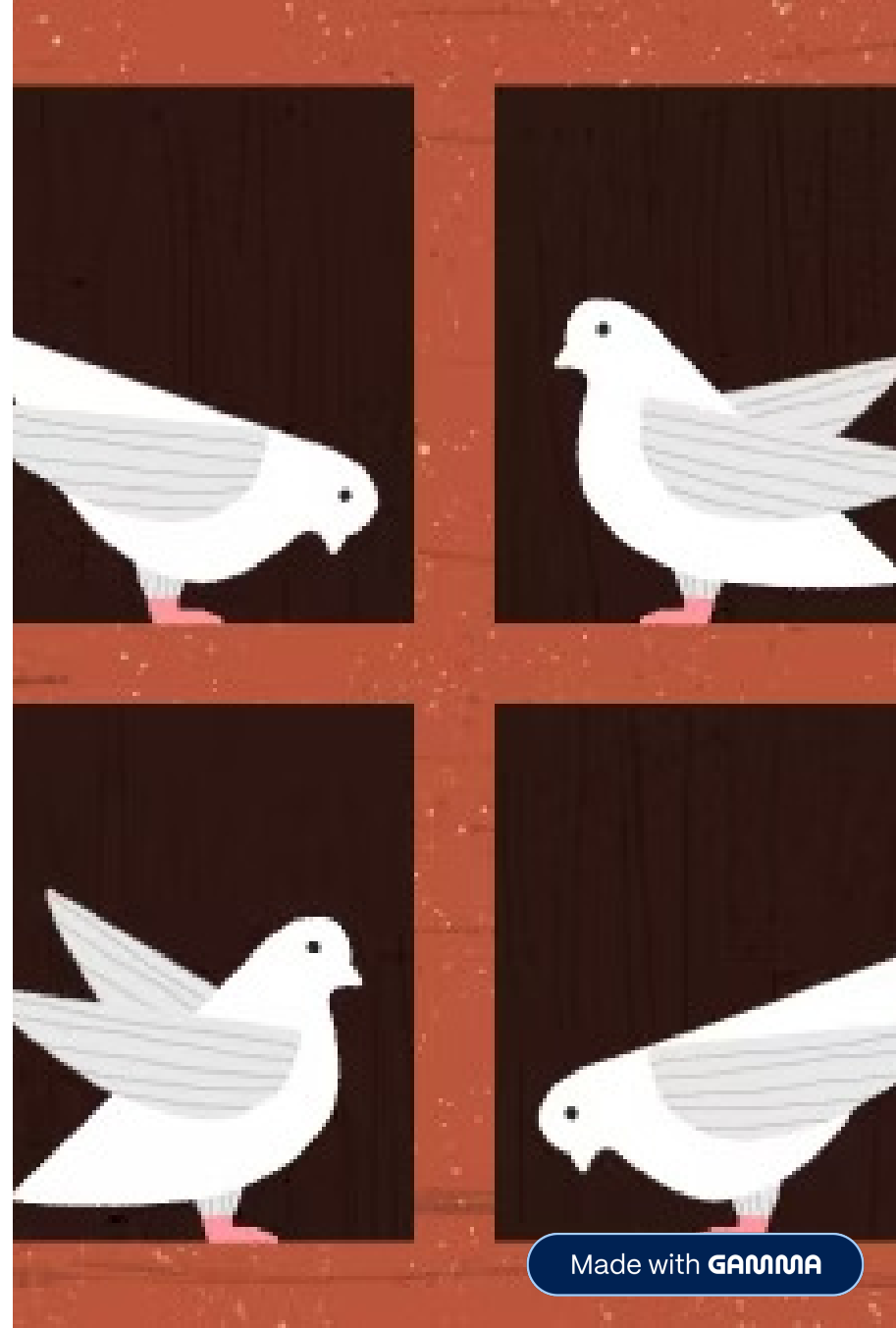


Estrutura de Dados Lineares.

Pigeonhole Sort (Caixa de Pombos)

*Um algoritmo de ordenação **não**-comparativo.*

Por **Lucas Vinícius** e **Maycon Alexsander**



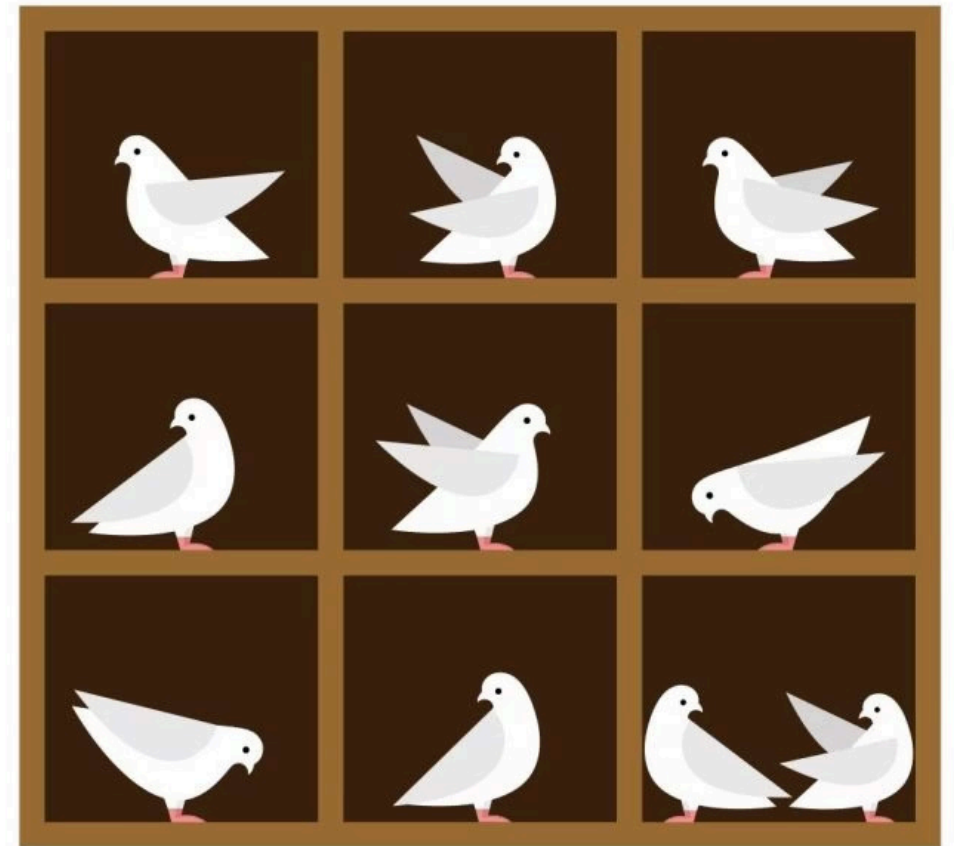
Introdução sobre o Método

O que é?

Diferente do Bubble ou Quick Sort, ele **não** usa comparações entre elementos (sem $\text{if } A > B$).

A Teoria

Baseado no "Princípio da Caixa de Pombos" (Se há mais pombos que ninhos, um ninho terá mais de um pombo).



Como funciona

Preparação

- Array inicial: [8, 3, 2, 7, 4, 6, 8]
- Acha-se o **Menor (2)** e o **Maior (8)**.
- Cria-se um array de caixas vazias. Tamanho = $(8 - 2) + 1 = 7$.

Mapeamento

- **Índice = Número - Min.**
- Varremos o array e somamos +1 na caixa correspondente.
- *Resultado nas caixas:* [1, 1, 1, 0, 1, 1, 2] (A última caixa tem "2" porque o 8 repetiu).

Recolhimento

- **Valor = Índice da Caixa + Min.**
- Varremos as caixas e sobrescrevemos o array original sequencialmente.
- *Array Final Ordenado:* [2, 3, 4, 6, 7, 8, 8].

Código

```
public static void pigeonholeSort(int[] array) {  
    // Array nulo ou com tamanho 0/1: ORDENADO  
    if (array == null || array.length <= 1) return;  
  
    // Descobrir menor e maior valor  
    int min = array[0], max = array[0];  
    for (int i = 1; i < array.length; i++) {  
        if (array[i] < min) min = array[i];  
        if (array[i] > max) max = array[i];  
    }  
  
    int[] caixas = new int[(max - min) + 1]; // armazena quantos pombos tem em cada caixa  
  
    // Contar pombos nas caixas  
    for (int i = 0; i < array.length; i++) {  
        caixas[array[i] - min]++; // +1 pombo  
    }  
  
    int indiceOriginal = 0;  
  
    // Ordenar o array  
    for (int i = 0; i < caixas.length; i++) {  
        while(caixas[i] > 0) {  
            array[indiceOriginal++] = i + min;  
            caixas[i]--;  
        }  
    }  
}
```

Análise Big O

Complexidade

Considerando **n** = total de números e **N** = tamanho das caixas (Range).

Tempo: $O(n + N)$. É um algoritmo de tempo linear.

Espaço: $O(N)$. Exige alocação de memória extra proporcional à diferença entre o maior e o menor número.

Pior Caso

O que acontece ao ordenar apenas 3 números: [1, 2, 999999]?

O computador criará um array inútil com **1 milhão de caixas** na memória RAM para ordenar 3 elementos. O N gigante destrói a performance e a memória.

Obrigado!



GitHub



GitHub – mayconalex/Pigeonhol...

Implementação manual do método de
ordenação PigeonholeSort –...